université PARIS-SACLAY | FACULTÉ DES SCIENCES D'ORSAY

LISN LABORATOIRE INTERDISCIPLINAIRE DES SCIENCES DU NUMÉRIQUE

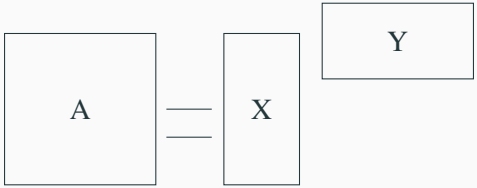
Mixed precision randomized low-rank approximation with GPU tensor cores

Matthieu Robeyns

Joint work with Marc Baboulin, Simplice Donfack, Oguz Kaya, and Theo Mary

Euro-Par 2024, August 28, 2024

Context



Low-rank approximation :

- Low-rank approximation is a key technique in numerical linear algebra to reduce the size of large matrices.
- Their computation is often the bottleneck in many applications.
- Randomized projection methods offer a good trade-off between accuracy and computational cost.

Goal : We aim to accelerate the computation of randomized low-rank approximations using mixed precision arithmetic and GPU tensor cores.

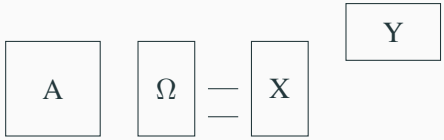
Randomized Low-rank approximation

Randomized low-rank approximation.

Input : $A \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X \in \mathbb{R}^{m \times k}$, $Y \in \mathbb{R}^{n \times k}$ s.t. $A \approx XY^T$.

1 : $\Omega \leftarrow \texttt{randn}(n, k+p)$
2 : $B \leftarrow A\Omega$
3 : $Q \leftarrow \texttt{qr}(B)$
4 : $Y \leftarrow A^T Q$
5 : $X = Q(:, 1:k)$
6 : $Y = Y(:, 1:k)$



Randomized low-rank approximation :

- **Randomized LRA** based on fixed rank Gaussian sampling.
- Bottleneck is the **matrix-matrix multiplication** (GEMM) on line 2 and 4 with $O(mnk)$ flops.

$\Rightarrow$ Accelerate the GEMM with **GPU tensor cores (TC)**.

Motivation

		Range	Precision	Speed(Tflop/s)*
fp64	double	$10^{\pm 308}$	1×10^{-16}	19.5
fp32	single	$10^{\pm 38}$	6×10^{-8}	19.5
fp16	half	$10^{\pm 5}$	5×10^{-4}	312
bfloat16		$10^{\pm 38}$	4×10^{-3}	

$\times 16.4$

*on NVIDIA A100 PCIe

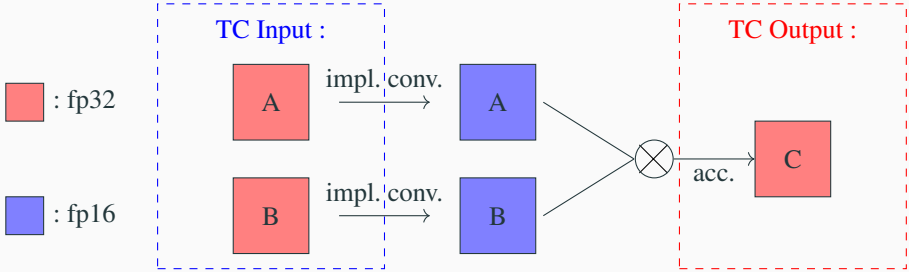
- Thanks to GPU tensor core, low precision is faster by a factor of 16.4 compare to high precision.
- Idea : low precision for high computational cost operations and high precision for the rest.

`tgemm` variants

- $\texttt{tgemm}_{32|32}$: A, B, C stored in fp32. A, B **implicitly** converted to fp16. C accumulated in fp32.
- $\texttt{tgemm}_{16|16}$: A, B, C stored in fp16 (**explicitly** converted before). C accumulated in fp16.
- $\texttt{tgemm}_{16|32}$: A, B stored in fp16 (**explicitly** converted before). C accumulated in fp32.

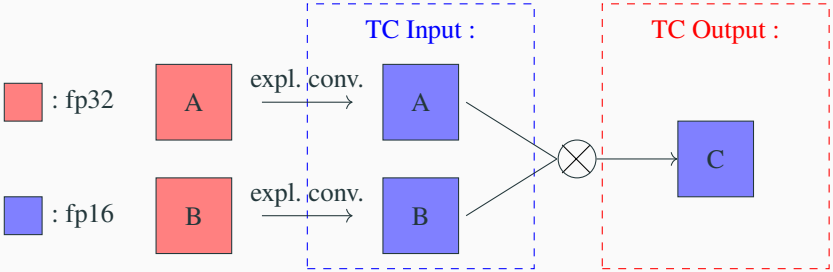
tgemm variants

- $\texttt{tgemm}_{32|32}$: A, B, C stored in fp32. A, B **implicitly** converted to fp16. C accumulated in fp32.
- $\texttt{tgemm}_{16|16}$: A, B, C stored in fp16 (**explicitly** converted before). C accumulated in fp16.
- $\texttt{tgemm}_{16|32}$: A, B stored in fp16 (**explicitly** converted before). C accumulated in fp32.



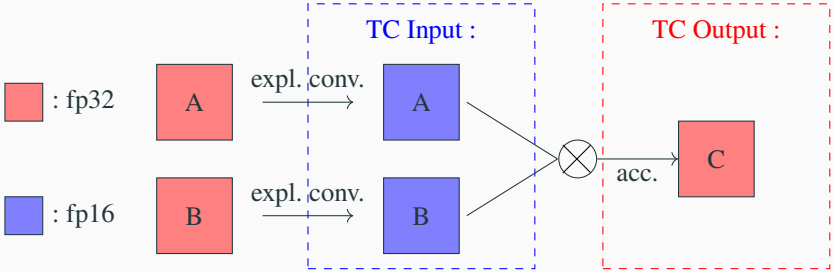
tgemm variants

- $\texttt{tgemm}_{32|32}$: A, B, C stored in fp32. A, B **implicitly** converted to fp16. C accumulated in fp32.
- $\texttt{tgemm}_{16|16}$: A, B, C stored in fp16 (**explicitly** converted before). C accumulated in fp16.
- $\texttt{tgemm}_{16|32}$: A, B stored in fp16 (**explicitly** converted before). C accumulated in fp32.

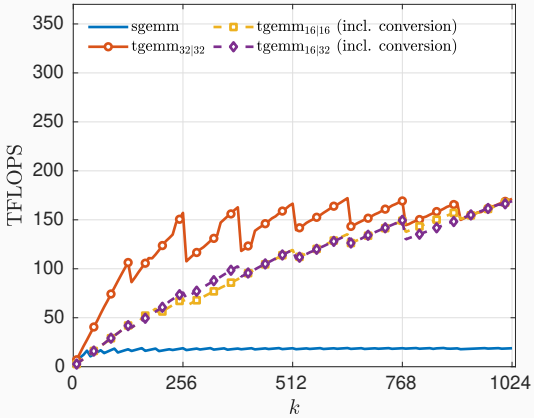


tgemm variants

- $\texttt{tgemm}_{32|32}$: A, B, C stored in fp32. A, B **implicitly** converted to fp16. C accumulated in fp32.
- $\texttt{tgemm}_{16|16}$: A, B, C stored in fp16 (**explicitly** converted before). C accumulated in fp16.
- $\texttt{tgemm}_{16|32}$: A, B stored in fp16 (**explicitly** converted before). C accumulated in fp32.

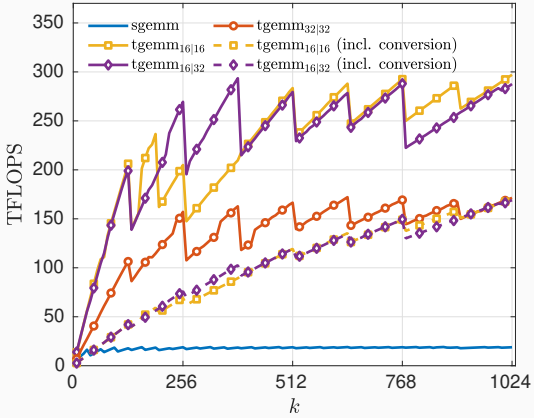


Performance of `tgemm` variants



- `tgemm`$_{32|32}$'s implicit conversion is more efficient than our explicit one.

Performance of `tgemm` variants



- $\texttt{tgemm}_{32|32}$'s implicit conversion is more efficient than our explicit one.
- However explicit conversion can be performed only once for multiple GEMMs.

Randomized low-rank approximation.

Input : $A \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X \in \mathbb{R}^{m \times k}$, $Y \in \mathbb{R}^{n \times k}$ s.t. $A \approx XY^T$.

1 : $\Omega \leftarrow \texttt{randn}(n, k+p)$

2 : $B \leftarrow A\Omega$

3 : $Q \leftarrow \texttt{qr}(B)$

4 : $Y \leftarrow A^T Q$

5 : $X = Q(:, 1:k)$

6 : $Y = Y(:, 1:k)$

Mixed precision randLRA with $\texttt{tgemm}_{16|32}$

Mixed precision randLRA on GPU tensor cores with $\texttt{tgemm}_{16|32}$.

Input : $A_{32} \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X_{16} \in \mathbb{R}^{m \times k}$ and $Y_{16} \in \mathbb{R}^{n \times k}$ such that $A_{32} \approx X_{16} Y_{16}^T$.

1: $\Omega_{16} = \texttt{randn}(n, k+p)$ [1]
2: $A_{16} = \texttt{fp16}(A_{32})$
3: $B_{32} = \texttt{tgemm}_{16|32}(A_{16}, \Omega_{16})$
4: $Q_{32} = \texttt{qr}(B_{32})$
5: $Q_{16} = \texttt{fp16}(Q_{32})$
6: $Y_{32} = \texttt{tgemm}_{16|32}(A_{16}^T, Q_{16})$
7: $X_{16} = \texttt{fp16}(Q_{32}(:, 1:k))$
8: $Y_{16} = \texttt{fp16}(Y_{32}(:, 1:k))$

1. *Mixed-Precision Random Projection for RandNLA on Tensor Cores*, Ootomo and Yokota (2023).

Mixed precision randLRA with $\texttt{tgemm}_{16|32}$

Mixed precision randLRA on GPU tensor cores with $\texttt{tgemm}_{16|32}$.

Input : $A_{32} \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X_{16} \in \mathbb{R}^{m \times k}$ and $Y_{16} \in \mathbb{R}^{n \times k}$ such that $A_{32} \approx X_{16} Y_{16}^T$.

1: $\Omega_{16} = \texttt{randn}(n, k+p)$ [1]
2: $A_{16} = \texttt{fp16}(A_{32})$
3: $B_{32} = \texttt{tgemm}_{16|32}(A_{16}, \Omega_{16})$
4: $Q_{32} = \texttt{qr}(B_{32})$ ⟵ Bottleneck now !
5: $Q_{16} = \texttt{fp16}(Q_{32})$
6: $Y_{32} = \texttt{tgemm}_{16|32}(A_{16}^T, Q_{16})$
7: $X_{16} = \texttt{fp16}(Q_{32}(:, 1:k))$
8: $Y_{16} = \texttt{fp16}(Y_{32}(:, 1:k))$

1. *Mixed-Precision Random Projection for RandNLA on Tensor Cores*, Ootomo and Yokota (2023).

Householder QR and Cholesky QR

Householder QR :

- 👍 : Perfectly stable.
- 👎 : Very slow, especially on GPU.

Householder QR and Cholesky QR

Householder QR :

- 👍 : Perfectly stable.
- 👎 : Very slow, especially on GPU.

Cholesky QR kernel.

Input : $A \in \mathbb{R}^{m \times n}$.

Output : Orthonormal factor $Q \in \mathbb{R}^{m \times n}$ of A.

1 : $B = A^T A$

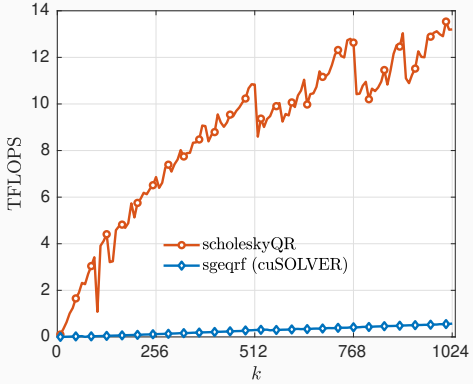
2 : $R = \texttt{chol}(B)$

3 : $Q = AR^{-1}$

Cholesky QR :

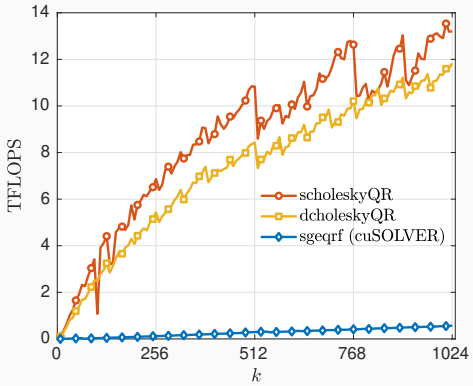
- 👍 : Based on GEMM $\Rightarrow$ very efficient on GPU.
- 👎 : Unstable, loss of orthogonality $\propto \kappa(B) = \kappa(A)^2$.

Performance of Cholesky QR kernel



- Cholesky QR up to 23× faster than Householder QR.
- About 14% of breakdowns with Cholesky QR in fp32.

Performance of Cholesky QR kernel



- Cholesky QR up to 23× faster than Householder QR.
- About 14% of breakdowns with Cholesky QR in fp32.
- Switching from fp32 to fp64 removes breakdowns and maintains almost the same performance.

Mixed precision randLRA with $\texttt{tgemm}_{16|32}$ and Cholesky QR

Mixed precision randLRA on GPU tensor cores with $\texttt{tgemm}_{16|32}$.

Input : $A_{32} \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X_{16} \in \mathbb{R}^{m \times k}$ and $Y_{16} \in \mathbb{R}^{n \times k}$ such that $A_{32} \approx X_{16} Y_{16}^T$.

1: $\Omega_{16} = \texttt{randn}(n, k+p)$ [1]
2: $A_{16} = \texttt{fp16}(A_{32})$
3: $B_{32} = \texttt{tgemm}_{16|32}(A_{16}, \Omega_{16})$
4: $Q_{32} = \texttt{qr}(B_{32})$
5: $Q_{16} = \texttt{fp16}(Q_{32})$
6: $Y_{32} = \texttt{tgemm}_{16|32}(A_{16}^T, Q_{16})$
7: $X_{16} = \texttt{fp16}(Q_{32}(:, 1:k))$
8: $Y_{16} = \texttt{fp16}(Y_{32}(:, 1:k))$

1. *Mixed-Precision Random Projection for RandNLA on Tensor Cores*, Ootomo and Yokota (2023).

Mixed precision randLRA with $\texttt{tgemm}_{16|32}$ and Cholesky QR

Mixed precision randLRA on GPU tensor cores.

Input : $A_{32} \in \mathbb{R}^{m\times n}$, the target rank k, the oversampling p.

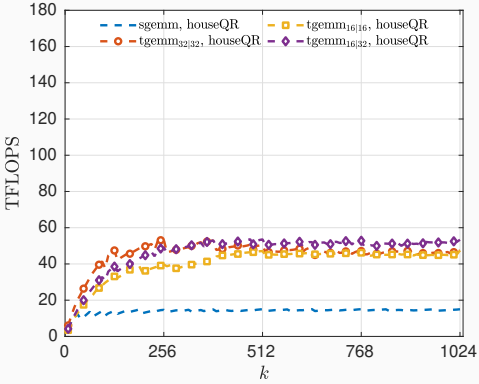
Output : $X_{16} \in \mathbb{R}^{m\times k}$ and $Y_{16} \in \mathbb{R}^{n\times k}$ such that $A_{32} \approx X_{16}Y_{16}^T$.

1 : $\Omega_{16} = \texttt{randn}(n, k+p)$
2 : $A_{16} = \texttt{fp16}(A_{32})$
3 : $B_{32} = \texttt{tgemm}_{16|32}(A_{16}, \Omega_{16})$

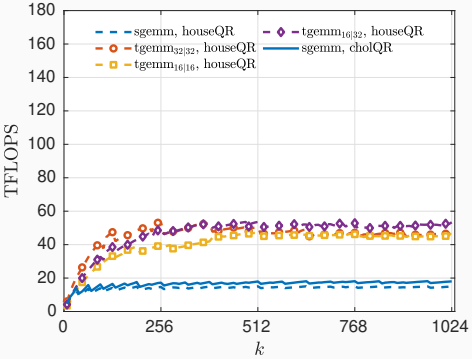
4 : $B_{64} = \texttt{fp64}(B_{32})$
5 : $C_{64} = B_{64}^T B_{64}$
6 : $R_{64} = \texttt{chol}(C_{64})$
7 : $Q_{64} = B_{64}R_{64}^{-1}$

8 : $Q_{16} = \texttt{fp16}(Q_{64})$
9 : $Y_{32} = \texttt{tgemm}_{16|32}(A_{16}^T, Q_{16})$
10 : $X_{16} = \texttt{fp16}(Q_{32}(:, 1{:}\,k))$
11 : $Y_{16} = \texttt{fp16}(Y_{32}(:, 1{:}\,k))$

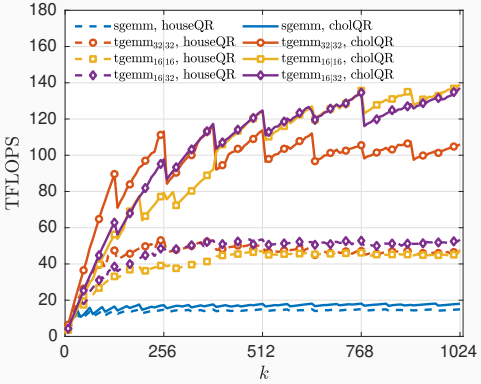
Performance of RandSVD kernels



Performance of RandSVD kernels

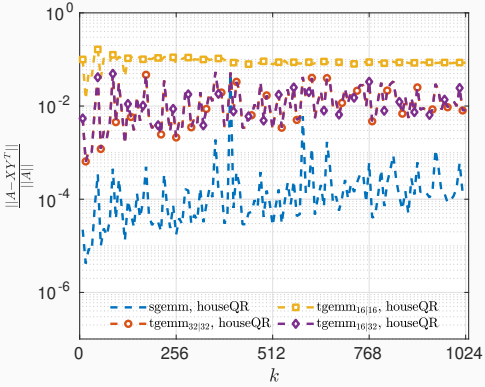


Performance of RandSVD kernels

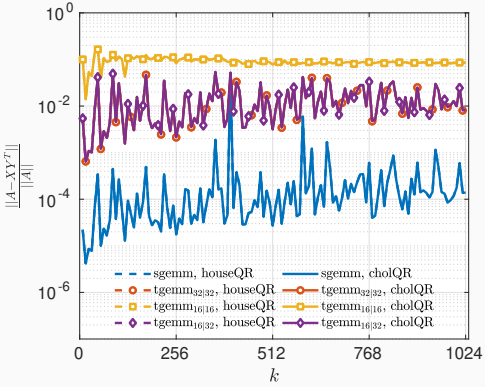


- RandSVD with $\texttt{tgemm}_{16|32}$ and Cholesky QR is $\times 9.1$ faster than SGEMM with Householder QR.
- $\texttt{tgemm}_{32|32}$ is very efficient for small size and slower than $\texttt{tgemm}_{16|32}$ for large size.

Accuracy of RandSVD kernels



Accuracy of RandSVD kernels



- Cholesky QR had same accuracy than Householder QR.
- Still less accurate of two orders of magnitude by using `tgemm`.

Iterative refinement

Randomized LRA with iterative refinement[1].

Input : $A \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X \in \mathbb{R}^{m \times 3k}$ and $Y \in \mathbb{R}^{n \times 3k}$ such that $A \approx XY^T$.

1 : $[X_1, Y_1] = \texttt{randLRA}(A, k, p)$

2 : $E = A - X_1 Y_1^T$

3 : $[X_2, Y_2] = \texttt{randLRA}(E, 2k, p)$

4 : $X = [X_1, X_2]$

5 : $Y = [Y_1, Y_2]$

1. *Mixed precision iterative refinement for low-rank matrix and tensor approximations*, Baboulin et al. (2023).

Iterative refinement

Mixed precision `randLRA` on GPU tensor cores, with iterative refinement[1].

Input : $A_{32} \in \mathbb{R}^{m \times n}$, the target rank k, the oversampling p.

Output : $X_{16} \in \mathbb{R}^{m \times 3k}$ and $Y_{16} \in \mathbb{R}^{n \times 3k}$ such that $A_{32} \approx X_{16} Y_{16}^T$.

1 : $[X_{16}, Y_{16}] = \texttt{randLRA}(A_{32}, k, p)$

2 : $E_{32} = A_{32} - \texttt{tgemm}_{16|32}(X_{16}, Y_{16}^T)$

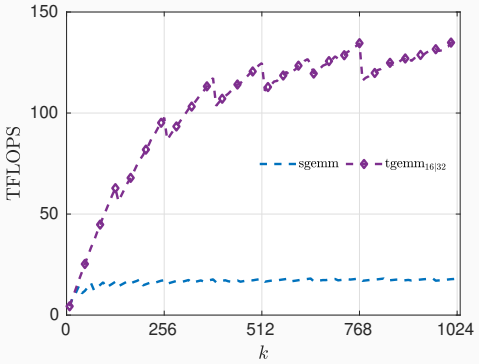
3 : $[X'_{16}, Y'_{16}] = \texttt{randLRA}(E_{32}, 2k, p)$

4 : $X_{16} = [X_{16}, X'_{16}]$

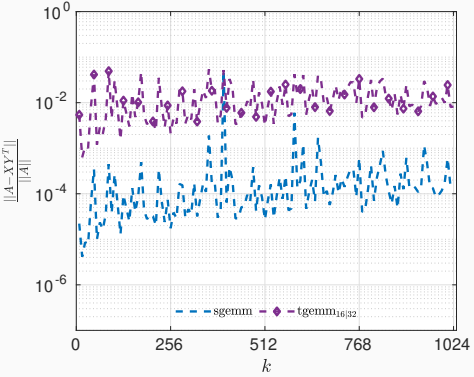
5 : $Y_{16} = [Y_{16}, Y'_{16}]$

1. *Mixed precision iterative refinement for low-rank matrix and tensor approximations*, Baboulin et al. (2023).

Performance and accuracy of randLRA *with* refinement and Cholesky QR kernel

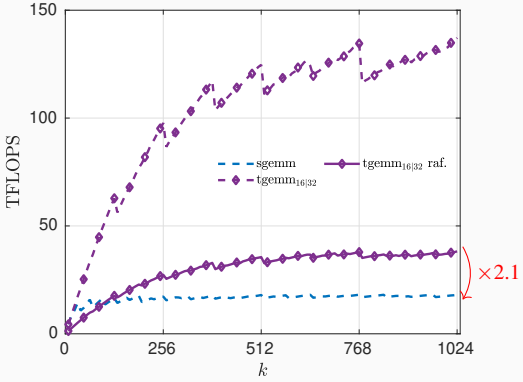


Performance

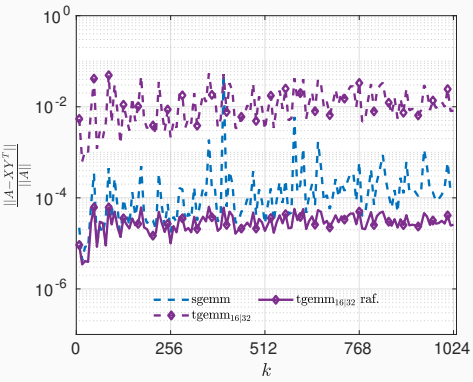


Accuracy

Performance and accuracy of randLRA *with* refinement and Cholesky QR kernel



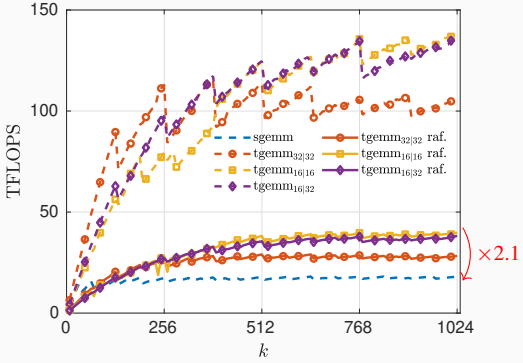
Performance



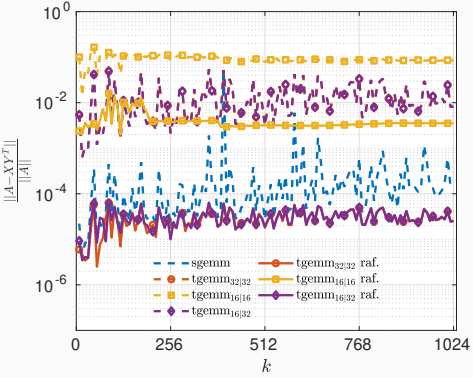
Accuracy

- randLRA with refinement is $\times 2.1$ faster than SGEMM.
- Also more accurate than SGEMM.

Performance and accuracy of randLRA *with* refinement and Cholesky QR kernel



Performance



Accuracy

- randLRA with refinement is ×2.1 faster than SGEMM.
- Also more accurate than SGEMM.

Conclusion

Contribution

A mixed precision randomized low-rank approximation algorithm :

- GEMM kernel accelerated with **GPU** tensor cores (fp16/fp32).
- QR kernel accelerated with **Cholesky QR** stabilized using fp64.
- fp32 accuracy recovered with **iterative refinement**.

Perspectives

- Mixed precision Cholesky QR kernel for GPU tensor cores.
- Efficient recompression method to reduce rank from $3k$ to k.

THANK YOU ! Questions ?